

CODEFIXER AI

KOMPLEKSOWA DOKUMENTACJA PROJEKTOWA: PRD, ARD & PLAN SDLC

Niniejszy zestaw dokumentów stanowi formalną specyfikację techniczną i funkcjonalną dla inteligentnego systemu asystenckiego **CodeFixer AI**, dedykowanego do zaawansowanego debugowania, analizy logicznej oraz automatycznej korekty kodu źródłowego. System wykorzystuje architekturę wieloagentową z implementacją modeli LRM (Large Reasoning Models) w celu zapewnienia najwyższej precyzji wnioskowania rezolucyjnego.

1. PRD (Product Requirements Document) – Dokument Wymagań Produktowych

1.1. Cel i wizja produktu

CodeFixer AI to zaawansowany system czatu inżynierskiego, którego celem jest skrócenie czasu usuwania awarii (MTTR) w procesie wytwarzania oprogramowania. Użytkownik wprowadza napotkany błąd programistyczny oraz powiązany fragment kodu, a autonomiczny system agentów AI przeprowadza wieloetapowe wnioskowanie (Reasoning), wyszukuje optymalne rozwiązanie, a następnie testuje i dostarcza gotową, zweryfikowaną strukturalnie poprawkę. System łączy formalną estetykę instytucjonalną z najnowocześniejszymi paradygmatami sztucznej inteligencji.

1.2. Grupa docelowa (Persony)

- Inżynier Oprogramowania (Wytwórca):** Skupiony na szybkości działania. Potrzebuje precyzyjnej odpowiedzi "tu i teraz", bez przeszukiwania setek wątków na forach technologicznych. Wkleja błędy kompilacji lub wyjątków runtime i oczekuje gotowego kodu o zerowym współczynniku regresji.
- Architekt / Audytor Kodu (Quality Gatekeeper):** Wymaga transparentności. Chce widzieć ścieżkę rozumowania modelu (Chain-of-Thought), aby upewnić się, że zaproponowana poprawka nie generuje luk bezpieczeństwa i jest zgodna z wzorcami projektowymi.

1.3. Kluczowe funkcjonalności (Features) i wymagania

- Interfejs Konwersacyjny LRM:** Czat obsługujący zaawansowane tokeny myślowe. Wyświetla proces dedukcyjny przed podaniem ostatecznego wyniku.
- Autonomiczny Weryfikator Środowiskowy (OpenCode Worker):** Robot wykonawczy uruchamiany w izolowanym kontenerze, testujący poprawność semantyczną generowanego kodu.
- Kaskadowe Zarządzanie Modelami (Fallbacks):** Automatyczne przełączanie kontekstu LLM pomiędzy głównym modelem Hermes 3 (poprzez bramę OpenRouter) a systemami rezerwowymi OpenAI Codex/ChatGPT w przypadku anomalii sieciowych lub przekroczenia limitów (Rate Limits).
- Instytucjonalna Wizualizacja:** Interfejs użytkownika zaprojektowany w ścisłej korelacji z kanonami estetycznymi Narodowego Banku Polskiego (nbp.pl), promujący ład architektoniczny, zaufanie oraz pełną dostępność cyfrową.

1.4. Konkretnie User Stories (Opowieści Użytkowników)

US-01: Wprowadzanie kodu i analiza błędów

Jako deweloper aplikacji, *chcę* wkleić do okna czatu nieskompilowany kod źródłowy wraz z pełnym śladem stosu błędów (Stack Trace), *aby* system natychmiastowo zidentyfikował dokładną linię kodu oraz przyczynę usterki.

Kryteria akceptacji: System automatycznie wykrywa język programowania (ze szczególnym uwzględnieniem Python, Java, JavaScript), parsuje log błędu, a czas odpowiedzi wstępnej nie przekracza **t < 1.2s**.

US-02: Przegląd ścieżki wnioskowania (Reasoning Flow)

Jako architekt systemu, *chcę* mieć wgląd w pełną, rozwiniętą strukturę myślową agenta AI (tzw. reasoning tokens), *aby* zrozumieć, dlaczego dana poprawka została uznana za optymalną pod kątem wydajnościowym.

Kryteria akceptacji: Interfejs udostępnia dedykowany, zwijany panel "Proces Myślowy Agent", sformatowany zgodnie z paletą kolorystyczną NBP, prezentujący etapy eliminacji hipotez błędnych.

US-03: Automatyczna walidacja kodu przez Agent Robotnika

Jako inżynier DevOps, *chcę*, aby wygenerowany przez system kod został automatycznie przetestowany w odizolowanym środowisku przez worker "OpenCode", *aby* mieć pewność, że proponowana zmiana nie zawiera błędów syntaktycznych.

Kryteria akceptacji: Kod jest uruchamiany w piaskownicy (sandbox). Wynik kompilacji lub testu jednostkowego (status sukces/porażka) jest zwracany do interfejsu czatu jako metadane systemowe.

US-04: Odporność na awarie dostawców LLM (Failover)

Jako menedżer utrzymania systemu, *chcę*, aby w przypadku niedostępności API OpenRouter (Hermes 3) zapytania były automatycznie i przezroczystie przekierowywane do OpenAI Codex (ChatGPT API), *aby* zagwarantować nieprzerwaną pracę zespołu deweloperskiego.

Kryteria akceptacji: Przełączenie następuje automatycznie w warstwie backendowej po dwóch nieudanych próbach połączenia (timeout **t > 3.0s**), logując incydent w bazie PostgreSQL.

1.5. Wymagania niefunkcjonalne

- **Wydajność:** Czas renderowania strumieniowego (token-by-token) rozpoczyna się poniżej 800 ms od wysłania zapytania.
- **Bezpieczeństwo:** Całkowita izolacja środowiska uruchomieniowego OpenCode. Kod użytkownika nie może uzyskać dostępu do zasobów hosta backendowego ani danych innych sesji.
- **Dostępność (SLA):** Ciągłość działania operacyjnego na poziomie 99.95% w ujęciu miesięcznym.

1.6. Kryteria sukcesu i metryki (KPI)

Identyfikator KPI	Nazwa Metryki	Wartość Docelowa (Target)	Metoda Pomiaru
KPI-01	Skuteczność Napraw (Fix Accuracy Ratio)	> 82% poprawnych kompilacji za 1. razem	Automatyczne logi z walidatora OpenCode

Identyfikator KPI	Nazwa Metryki	Wartość Docelowa (Target)	Metoda Pomiaru
KPI-02	Czas Rozwiązania Problemu (Mean Time to Resolution)	< 45 sekund na pełną pętlę naprawczą	Telemetria backendowa (Node.js Gateway)
KPI-03	Retencja Użytkowników Technicznych (Weekly Retention)	> 45% aktywnych programistów po 4 tygodniach	Analityka sesji zalogowanych w bazie PostgreSQL

2. ARD (Architectural Reference Document) – Architektoniczny Dokument Referencyjny

2.1. Koncepcja Architektury Poliglotycznej

W celu optymalizacji obciążeń i wykorzystania natywnych ekosystemów technologicznych, system został zaprojektowany w oparciu o architekturę mikrousług skorelowanych funkcjonalnie:

Komponent / Warstwa	Technologia główna	Uzasadnienie Architektoniczne i Rola w Systemie
Frontend i Interfejs Użytkownika	React.js / TypeScript	Zapewnia dynamiczne zarządzanie stanem czatu i renderowanie strumieniowe (SSE). Ostylowany ściśle według wytycznych identyfikacji wizualnej NBP.pl.
BFF / Gateway Orkiestracyjny	Node.js / Express	Warstwa pośrednicząca (Backend-For-Frontend). Odpowiada za autoryzację, zarządzanie sesjami użytkowników, szybkie trasowanie zapytań HTTP/WebSocket oraz asynchroniczny zapis logów audytowych.
Rdzeń Agentowy AI (Orchestrator)	Python / LangGraph / LangChain	Serce systemu odpowiedzialne za grafowe modelowanie cyklu życia agentów. Kontroluje pętle wnioskowania (Stateful Multi-Agent Loops), zarządza pamięcią konwersacyjną i podejmuje decyzje o routingu LLM.
Brama Integracji Enterprise	Java / LangChain4j / Spring Boot	Komponent odpowiedzialny za integrację z korporacyjnymi repozytoriami kodu, statyczną analizę kodu (AST parsing) oraz bezpieczne interfejsowanie z systemami typu legacy z wykorzystaniem silnych mechanizmów typowania Java.
Baza Danych	PostgreSQL (v16)	Relacyjna baza danych zapewniająca pełną zgodność z zasadami ACID. Przechowuje profile użytkowników, historię czatów, szczegółowe logi wywołań LLM oraz metryki wykonania kodu.

2.2. Silnik Sztucznej Inteligencji i Topologia Agentowa

Logika wnioskowania zorganizowana jest w acykliczny graf skierowany (DAG) zarządzany przez `LangGraph`. Architektura wyróżnia następujące role agentowe:

- **Agent Konsultujący (Router):** Analizuje kod użytkownika i decyduje, do jakiego podgrafu skierować zapytanie w zależności od wykrytego języka programowania.
- **Agent Wnioskujący (Reasoning Agent):** Wykorzystuje model **Hermes 3 (OpenHermes)** za pośrednictwem tokenów **OpenRouter**. Model ten generuje strukturę tokenów myślowych, rozbijając błąd na czynniki pierwsze. W przypadku przekroczenia limitów, warstwa abstrakcji `LangChain` przełącza zapytanie na model **ChatGPT / OpenAI Codex**.
- **Agent Robotnik (OpenCode Execution Worker):** Wykorzystuje wyspecjalizowany model/środowisko uruchomieniowe do egzekucji kodu w bezpiecznej piaskownicy. Zwraca surowe logi wyjściowe i kody błędów (**exit codes**) z powrotem do Agenta Wnioskującego w celu ewentualnej autorefleksji (Self-Correction Loop).

2.3. Wytyczne Identyfikacji Wizualnej (Stylistyka NBP.pl)

Warstwa prezentacji systemu CodeFixer AI odrzuca generyczne szablony technologiczne na rzecz autorytarnego i eleganckiego wzornictwa nawiązującego do Narodowego Banku Polskiego. Zasady kompozycji obejmują:

- **Paleta Kolorystyczna:** Kolorem dominującym jest głęboki granat instytucjonalny (**#002C5B**), symbolizujący stabilność i bezpieczeństwo. Akcenty, obramowania wyróżniające oraz nagłówki niższego rzędu wykorzystują stonowane złoto (**#B59A57**). Tło aplikacji przyjmuje odcień matowego kremu/jasnej szarości (**#FCFBFA**).
- **Oznaczenia i Godło:** Logo systemu CodeFixer AI jest wpisane w geometryczny, centralnie umieszczony sygnet z wyraźną, klasyczną typografią szeryfową. Wszystkie sekcje informacyjne i tabele posiadają wyraźne, cienkie linie podziału, nawiązujące do układu raportów finansowych.

3. PLAN (Plan Projektowy i Harmonogram SDLC)

Projekt prowadzony będzie w zwinnej metodyce Scrum w ramach 6-miesięcznego, rygorystycznego cyklu życia oprogramowania (SDLC). Każdy sprint trwa dokładnie 2 tygodnie.

3.1. Faza 1: Inicjacja, Projektowanie i Architektura (Miesiąc 1-2)

- **Analiza wymagań i UX/UI:** Przygotowanie pełnych makiet interfejsu czatu w programie Figma, uwzględniając restrykcyjne wytyczne kolorystyczne i typograficzne portalu NBP.pl.
- **Infrastruktura i Baza Danych:** Projektowanie schematu relacyjnego w bazie PostgreSQL. Konfiguracja klastra deweloperskiego.
- **Inicjalizacja Szkieletu Poliglotycznego:** Skonfigurowanie repozytorium monorepo. Uruchomienie aplikacji bazowej w Node.js (Gateway), Pythonie (LangChain) oraz Javie (LangChain4j).

3.2. Faza 2: Faza Wdrożeniowa i Sprints (Miesiąc 3-4)

Implementacja kluczowych domen biznesowych systemu w ramach iteracyjnych przyrostów:

Numery Sprintów	Obszar Technologiczny	Zakres Prac Programistycznych i Kamienie Milowe
Sprint 1 - 2	Infrastruktura, Gateway i PostgreSQL	Konfiguracja potoku CI/CD (GitHub Actions). Implementacja warstwy uwierzytelniania w Node.js. Utworzenie tabel sesji i audytu w PostgreSQL. Wykonanie struktury layoutu NBP (granat/złoto) we frontendzie React.
Sprint 3 - 4	Orkiestracja AI (Python & LangGraph)	Implementacja grafu agentowego w Pythonie przy użyciu LangGraph. Integracja z OpenRouter (Hermes 3) oraz mechanizmu failover do OpenAI Codex za pomocą abstrakcji LangChain. Strumieniowanie tokenów myślowych do frontendu.
Sprint 5 - 6	Java Gateway & OpenCode Integration	Wdrożenie mikrousługi w Javie z wykorzystaniem LangChain4j do zaawansowanego parsowania struktur kodu. Integracja z robotem wykonawczym OpenCode Worker. Zamknięcie pętli autorefleksji agenta (Self-Correction Loop).

3.3. Faza 3: Testy i Zapewnienie Jakości (Miesiąc 5)

- **Testy Jednostkowe i Integracyjne:** Osiągnięcie pokrycia kodu (Code Coverage) na poziomie minimum 85% dla krytycznych komponentów logicznych w Pythonie i Javie.
- **Testy Bezpieczeństwa (Penetration Testing):** Weryfikacja szczelności kontenerów izolacyjnych OpenCode pod kątem podatności typu Remote Code Execution (RCE).
- **Zamknięte Testy Beta:** Uruchomienie platformy dla kohorty 500 inżynierów oprogramowania. Monitorowanie współczynnika utraty tokenów oraz stabilności mechanizmu failover.

3.4. Faza 4: Wdrożenie Produkcyjne i Utrzymanie (Miesiąc 6)

- **Wdrożenie Środowiska (Production Release):** Konteneryzacja całości systemu za pomocą Docker i wdrożenie na infrastrukturę Kubernetes.
- **Monitorowanie i Telemetria:** Spięcie logów aplikacyjnych Node.js, Python i Java z systemem Datadog oraz Firebase Performance w celu ciągłej kontroli opóźnień (latency tracking) generowania tokenów.
- **Przekazanie do Utrzymania:** Uruchomienie procedur wsparcia technicznego i zbieranie zgłoszeń dotyczących nierozpoznanych struktur błędów.